Unit 3

Contents:

R-Function :function definition,
Built in functions: mean(), paste(), sum(), min(), max(), seq(),
user-defined function, calling a function, calling a function without an argument, calling a function with argument values –
Manipulating Text in Data:substr(),strsplit(), paste(), grep(), toupper(), tolower() –
R Vectors–Sequence vector, rep function,vector access, vector names, vector math, vector recycling, vector element sorting –
R List -Creating a List, List Tags and Values, Add/Delete Element to or from a ListSize of List,Merging Lists, Converting List to Vector –
R Matrices –AccessingElements of a Matrix,Matrix Computations: Addition, subtraction,Multiplication and Division-
R Arrays: NamingColumns and Rows, AccessingArray Elements, Manipulating Array Elements, CalculationAcross ArrayElements –
R Factors–creating factors, generating factor levels gl().

1) R Functions

A set of statements which are organized together to perform a specific task is known as a function.

R provides a series of in-built functions, and it allows the user to create their own functions. Functions are used to perform tasks in the modular approach.

Functions are used **to avoid repeating the same task and to reduce complexity**. To understand and maintain our code, we logically break it into smaller parts using the function. A function should be

1. Written to carry out a specified task.
2. May or may not have arguments
3. Contain a body in which our code is written.
4. May or may not return one or more output values.

"An R function is created by using the keyword function."

There is the following syntax of R function:

```
func_name <- function(arg_1, arg_2, ...) {
        Function body
}
```

2) Function Types

Similar to the other languages, R also has two types of function, i.e. **Built-in Function** and **User-defined Function**. In R, there are lots of built-in functions which we can directly call in the program without defining them. R also allows us to create our own functions.

a) Built-in function

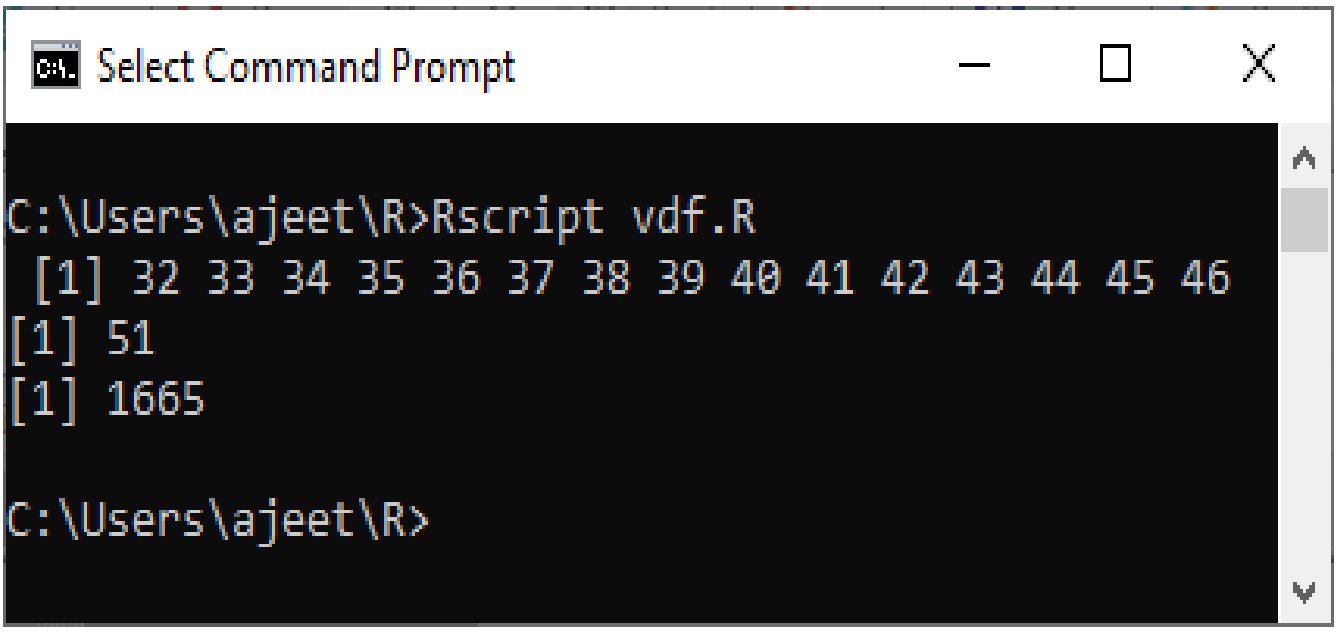
The functions which **are already created or defined in the programming framework are known as built-in functions.** User doesn't need to create these types of functions, and these functions are built into an application. End-users can access these functions by simply calling it.

R have different types of built-in functions such as seq(), mean(), max(), and sum(x) etc., paste(), , min(), max()

Example:

```
# Creating sequence of numbers from 32 to 46.
print(seq(32,46))
# Finding the mean of numbers from 22 to 80.
print(mean(22:80))
# Finding the sum of numbers from 41 to 70.
print(sum(41:70))
```

Output:



b) User-defined function

R allows us **to create our own function in our program**. A user defines a user-define function **to fulfill the requirement of user.** Once these functions are created, we can use these functions like in-built function.

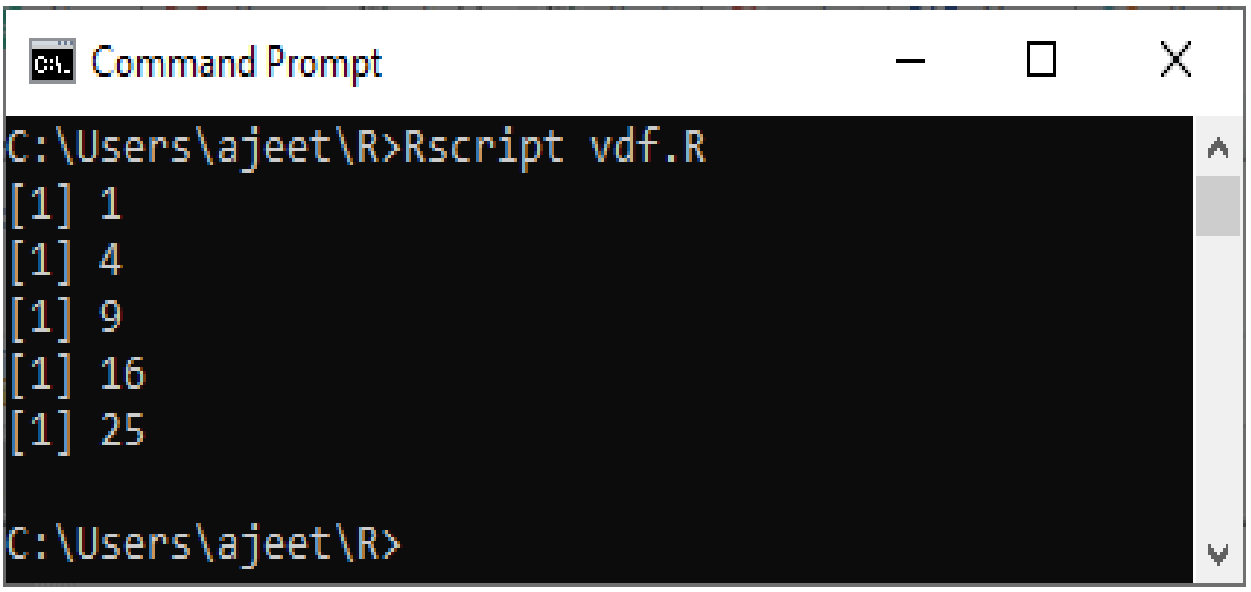
1) Creating a function without an argument.

Example:

```
# Creating a function without an argument.
new.function<- function() {
for(i in 1:5) {
    print(i^2)
  }
}

new.function()
```

Output:



2) Function calling with an argument

We can easily call a function by passing an appropriate argument in the function. Let see an example to see how a function is called.

```
# Creating a function to print squares of numbers in sequence.
new.function <- function(a) {
  for(i in 1:a) {
    b <- i^2
    print(b)
  }

# Calling the function new.function supplying 10 as an argument.
new.function(10)
```

Output:

```
Command Prompt
Microsoft Windows [Version 10.0.18362.239]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\ajeet>cd R

C:\Users\ajeet\R>Rscript calling.R
[1] 1
[1] 4
[1] 9
[1] 16
[1] 25
[1] 36
[1] 49
[1] 64
[1] 81
[1] 100

C:\Users\ajeet\R>
```

3) Write the functions available for string manipulation in R:

R provides various string functions to perform various tasks to manipulate the string.

These string functions allow us to extract sub string from string, search pattern etc. **Few string functions listed below :**

a) substr(),
b) strsplit(),
c) paste(),
d) grep(),
e) toupper(),
f) tolower()

1) Join Strings Together: paste()

In R, we can use the paste() function **to join two or more strings together**. For example

```
message1<- "Programiz"
message2 <- "Pro"

# use paste() to join two strings
paste(message1, message2)
```

OutPut:

```
[1] Programiz Pro
```

2) Change Case of R String

In R, we can change the case of a string using

- `toupper()` - convert string to uppercase
- `tolower()` - convert string to lowercase

```
message <- "R Programming"

# change string to uppercase
M1<- toupper(message)
cat("Uppercase:", M1)

# change string to lowercase
M2<- tolower(message)
cat("\nLowercase:", M2)
```

Output:

```
Uppercase: R PROGRAMMING
Lowercase: r programming
```

3)**strsplit () :** A string can **be split into corresponding individual strings using " " the default separator.**

Example:

strsplit("Learn Code Teach !", " ")

Output:

```
[1] "Learn" "Code" "Teach" "!"
```

4)**substr():***substr(..., start, end) or substring(..., start, end) function in R* ***extracts substrings out of a string beginning with the start index and ending with the end index.*** *It also replaces the specified substring with a new set of characters.*

Example:

```
Python3
substr("Learn Code Tech", 1, 4)
```

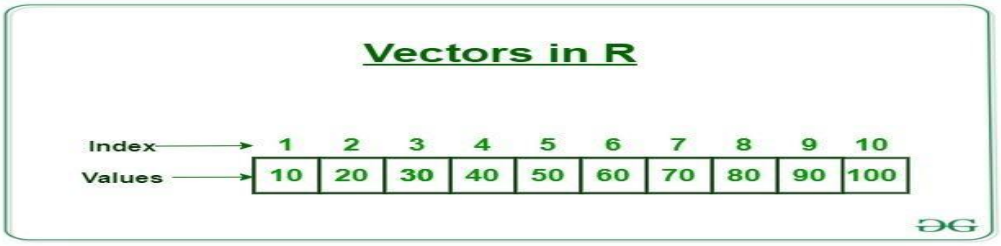
Output:

```
"Lear"
```

5)`grep()`:**grep()** return vector of indices of the element if a pattern exists in that vector.

4) R Vectors

R Vectors are the same as the arrays in R language which are used to hold multiple data values of the same type. One major key point is that in R Programming Language the indexing of the vector will start from '1' and not from '0'. We can create numeric vectors and character vectors as well.



How to create a vector in R?

In R, we use c() function to create a vector. This function returns a one-dimensional array or simply vector. The c() function is a generic function which combines its argument. All arguments are restricted with a common data type which is the type of the returned value. There are various other ways to create a vector in R, which are as follows:

1) Using the colon(:) operator

We can create a vector with the help of the colon operator. There is the following syntax to use colon operator:

1. z<-x:y

This operator creates a vector with elements from x to y and assigns it to z.

Example:

```
a<-4:-10
a
```

Output

```
[1]   4   3   2   1   0   -1   -2   -3   -4   -5   -6   -7   -8
```

2) Using the seq() function

In R, we can create a vector with the help of the seq() function. A sequence function creates a sequence of elements as a vector. The seq() function is used in two ways, i.e., by setting step size with ?by' parameter or specifying the length of the vector with the 'length.out' feature.

Example:

```
seq_vec<-seq(1,4,by=0.5)
seq_vec
class(seq_vec)
```

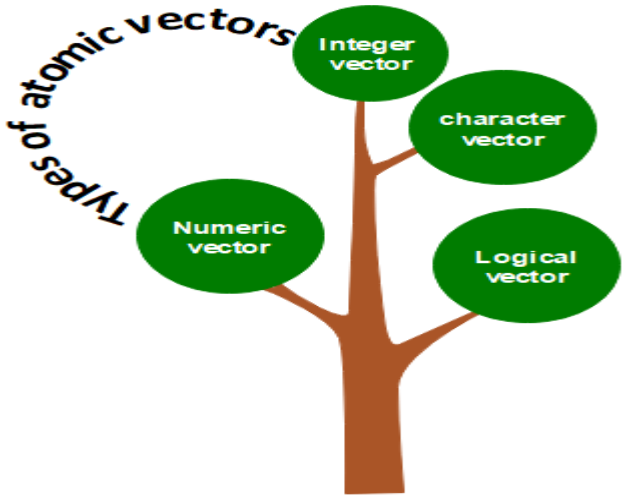
Output

```
[1]   1.0   1.5   2.0   2.5   3.0   3.5   4.0
```

Types of R vectors

Vectors are of different types which are used in R. Following are some of the types of vectors:

In R, there are four types of atomic vectors. Atomic vectors play an important role in Data Science. Atomic vectors are created with the help of **c()** function. These atomic vectors are as follows:



a) Numeric vector:Numeric vectors are those which **contain numeric values such as integer, float, etc.**

Example:

```
# R program to create numeric Vectors

# creation of vectors using c() function.
v1<- c(4, 5, 6, 7)

# display type of vector
typeof(v1)

# by using 'L' we can specify that we want integer values.
v2<- c(1L, 4L, 2L, 5L)

# display type of vector
typeof(v2)
```

Output:

```
[1] "double"
[1] "integer"
```

b) Integer vector

A non-fraction numeric value is known as integer data. There is two way to assign an integer value to a variable, i.e., by using **as.integer() function and appending of L** to the value.

c)Character vector

A character is held as a one-byte integer in memory. In R, there are two different ways to create a character data type value, i.e., using as.character() function and by typing string between double quotes("") or single quotes('').

Character vectors in R contain alphanumeric values and special characters.

Example:

```R
# R program to create Character Vectors

# by default numeric values
# are converted into characters
v1<- c('geeks', '2', 'hello', 57)

# Displaying type of vector
typeof(v1)
```

Output:

```
[1] "character"
```

d) Logical vector

The logical data types **have only two values i.e., True or False. These values are based on which condition is satisfied.**

A vector which contains Boolean values is known as the logical vector.

Logical vectors in R contain Boolean values such as TRUE, FALSE and NA for Null values.

```r
# R program to create Logical Vectors

# Creating logical vector
# using c() function
v1<- c(TRUE, FALSE, TRUE, NA)

# Displaying type of vector
typeof(v1)
```

Output:

```
[1] "logical"
```

5) Accessing elements of vectors:

Accessing elements in a vector is **the process of performing operation on an individual element of a vector.**

There are many ways through which we can access the elements of the vector. The most common **is using the '[ ]', symbol.**

Note: *Vectors in R are 1 based indexing unlike the normal C, python, etc format.*

Example:

 # accessing elements with an index number.

X<- c(2, 5, 18, 1, 12)

cat('Using Subscript operator', X[2], '\n')

Output:

Using Subscript operator 5

6) Sorting elements of a R Vector

sort() function is used with the help of which **we can sort the values in ascending or descending order.**

```
# R program to sort elements of a Vector

# Creation of Vector
X<- c(8, 2, 7, 1, 11, 2)

# Sort in ascending order
A<- sort(X)
cat('ascending order', A, '\n')

# sort in descending order
# by setting decreasing as TRUE
B<- sort(X, decreasing = TRUE)
cat('descending order', B)
```

Output:

```
ascending order 1  2  2  7  8 11
descending order 11  8  7  2  2  1
```

7) Vector Operation

In R, there are various operation which is performed on the vector. **We can add, subtract, multiply or divide two or more vectors from each other**. In data science, R plays an important role, and operations are required for data manipulation. There are the following types of operation which are performed on the vector.

Arithmetic operations

We can perform all the arithmetic operation on vectors. The arithmetic operations are performed member-by-member on vectors. We can add, subtract, multiply, or divide two vectors. Let see an example to understand how arithmetic operations are performed on vectors.

Example:

```
a<-c(1,3,5,7)
b<-c(2,4,6,8)
a+b
a-b
a/b
```

Output

```
[1]  3  7 11 15
[1] -1 -1 -1 -1
[1]  2 12 30 56
```

5) R Lists

In R, lists are the second type of vector.

Lists are the objects of R which contain **elements of different types such as number, vectors, string and another list inside it**.

It can also contain a function or a matrix as its elements.

A list is a data structure which has components of mixed data types.

We can say, a list is a generic vector which contains other objects. Following are the 5 main functions of List.

1) Lists creation: The process of creating a list is the same as a vector. In R, function list() which is used to create a list in R.

   A list avoid the drawback of the vector which is same data type. We can add the elements in the list of different data types.

2) Giving a name to list elements: There are only three steps to print the list data corresponding to the name:

   a) Creating a list.

   b) Assign a name to the list elements with the help of names() function.

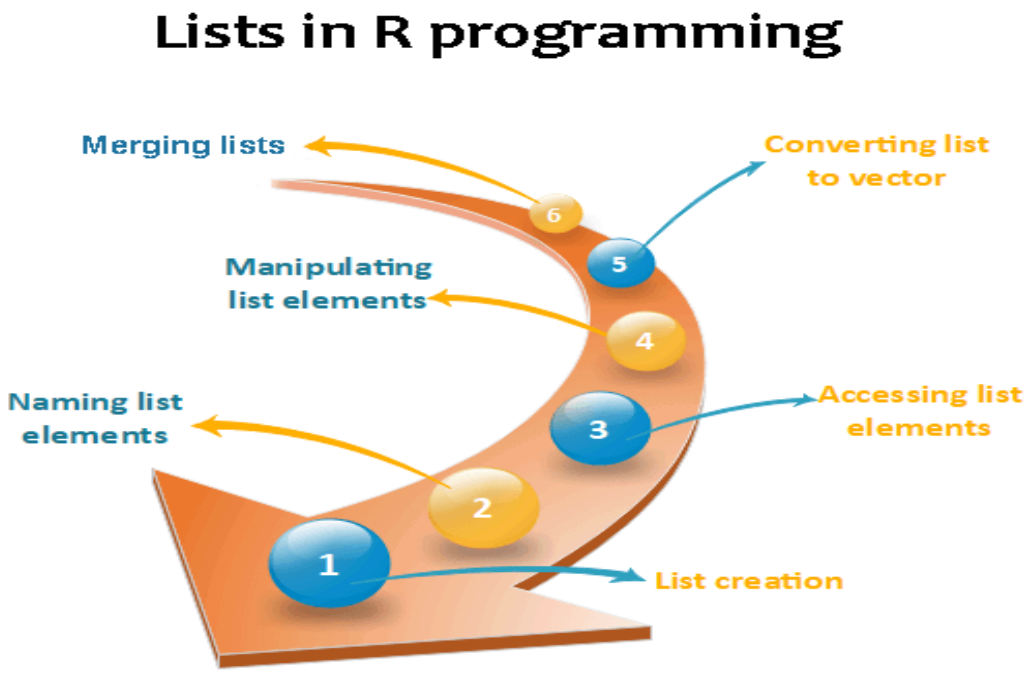
   c) Print the list data.

3) Accessing List Elements: Elements can be access by two methods:

   a) By using Indexing b) By using Names

4) Manipulation of list elements:R allows us to add, delete, or update elements in the list. We can update an element of a list from anywhere, but elements can add or delete only at the end of the list. To remove an element from a specified index, we will assign it a null value.

5) Converting list to vector:The unlist() function takes the list as a parameter and change into a vector.



Example1; WAP to create list and Access Elements from list.

```r
# R program to access
# components of a list

# Creating a list by naming all its components
empId = c(1, 2, 3, 4)
empName = c("Debi", "Sandeep", "Subham", "Shiba")
numberOfEmp = 4
empList = list(
  "ID" = empId,
  "Names" = empName,
  "Total Staff" = numberOfEmp
  )
print(empList)

# Accessing a top level components by indices
cat("Accessing name components using indices\n")
print(empList[[2]])

# Accessing a inner level components by indices
cat("Accessing Sandeep from name using indices\n")
print(empList[[2]][2])

# Accessing another inner level components by indices
cat("Accessing 4 from ID using indices\n")
print(empList[[1]][4])
```

Output:

```
$ID
[1] 1 2 3 4

$Names
[1] "Debi"    "Sandeep" "Subham"  "Shiba"

$`Total Staff`
[1] 4

Accessing name components using indices
[1] "Debi"    "Sandeep" "Subham"  "Shiba"
Accessing Sandeep from name using indices
[1] "Sandeep"
Accessing 4 from ID using indices
[1] 4
```

Example 2: WAP to Modify the List.

```
# R program to edit
# components of a list

# Creating a list by naming all its components
empId = c(1, 2, 3, 4)
empName = c("Debi", "Sandeep", "Subham", "Shiba")
numberOfEmp = 4
empList = list(
  "ID" = empId,
  "Names" = empName,
  "Total Staff" = numberOfEmp
)
cat("Before modifying the list\n")
print(empList)

# Modifying the top-level component
empList$`Total Staff` = 5

# Modifying inner level component
empList[[1]][5] = 5
empList[[2]][5] = "Kamala"

cat("After modified the list\n")
print(empList)
```

Output:

```
Before modifying the list
$ID
[1] 1 2 3 4

$Names
[1] "Debi"    "Sandeep" "Subham"  "Shiba"

$`Total Staff`
[1] 4

After modified the list
$ID
[1] 1 2 3 4 5

$Names
[1] "Debi"    "Sandeep" "Subham"  "Shiba"   "Kamala"

$`Total Staff`
[1] 5
```

R Matrices:

Matrix is a rectangular arrangement of numbers in rows and columns. In a matrix, as we know **rows are the ones that run horizontally** and **columns are the ones that run vertically.**

In R programming, matrices are two-dimensional, homogeneous data structures. These are some examples of matrices:

$$\begin{pmatrix} 1 & 5 & 3 \\ 4 & 9 & 2 \\ 5 & 6 & 7 \end{pmatrix} \qquad \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \qquad \begin{bmatrix} 1 & 4 & 5 \end{bmatrix}$$

R – Matrices

Creating a Matrix

To create a matrix in R you need to use the function called **matrix()**. The arguments to this **matrix()** are the set of elements in the vector. You have to

pass how many numbers of rows and how many numbers of columns you want to have in your matrix.

Note: *By default, matrices are in column-wise order.*

Example 1*: WAP to create a matrix*

```r
# R program to create a matrix

A = matrix(

  # Taking sequence of elements
  c(1, 2, 3, 4, 5, 6, 7, 8, 9),

  # No of rows
  nrow = 3,

  # No of columns
  ncol = 3,

  # By default matrices are in column-wise order
  # So this parameter decides how to arrange the matrix
  byrow = TRUE
)

# Naming rows
rownames(A) = c("a", "b", "c")

# Naming columns
colnames(A) = c("c", "d", "e")

cat("The 3x3 matrix:\n")
print(A)
```

Output:

```
The 3x3 matrix:
  c d e
a 1 2 3
b 4 5 6
c 7 8 9
```

Creating special matrices

R allows the creation of various different types of matrices with the use of arguments passed to the matrix() function.

Syntax: *matrix(k, m, n)*
Parameters:
k: *the constant*
m: *no of rows*
n: *no of columns*

Example: WAP to create special Matrices in R with k constant.

```
# R program to illustrate
# special matrices

# Matrix having 3 rows and 3 columns
# filled by a single constant 5
print(matrix(5, 3, 3))
```

Output:

```
     [,1] [,2] [,3]
[1,]    5    5    5
[2,]    5    5    5
[3,]    5    5    5
```

Example 2: WAP to add Matrices

```r
# R program to add two matrices

# Creating 1st Matrix
B = matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3)

# Creating 2nd Matrix
C = matrix(c(7, 8, 9, 10, 11, 12), nrow = 2, ncol = 3)

# Getting number of rows and columns
num_of_rows = nrow(B)
num_of_cols = ncol(B)

# Creating matrix to store results
sum = matrix(, nrow = num_of_rows, ncol = num_of_cols)

# Printing Original matrices
print(B)
print(C)
```

OutPut:

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
     [,1] [,2] [,3]
[1,]    7    9   11
[2,]    8   10   12
     [,1] [,2] [,3]
[1,]    8   12   16
[2,]   10   14   18
```

Example 2: WAP for Subtraction of Matrices

```r
# create a vector of elements
vector1=c(1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16)

# create a matrix with 4* 4 by passing this vector1
matrix1 <- matrix(vector1, nrow = 4, ncol = 4)

# display matrix
print(matrix1)

# create a vector of elements
vector2=c(1,2,3,2,4,5,6,3,4,1,2,7,8,9,4,5)

# create a matrix with 4* 4 by passing  vector2
matrix2 <- matrix(vector2, nrow = 4, ncol = 4)

# display matrix
print(matrix2)
print(" subtraction result")

# subtract matrices
print(matrix1-matrix2)
```

Output:

```
     [,1] [,2] [,3] [,4]
[1,]    1    5    9   13
[2,]    2    6   10   14
[3,]    3    7   11   15
[4,]    4    8   12   16
     [,1] [,2] [,3] [,4]
[1,]    1    4    4    8
[2,]    2    5    1    9
[3,]    3    6    2    4
[4,]    2    3    7    5
[1] " subtraction result"
     [,1] [,2] [,3] [,4]
[1,]    0    1    5    5
[2,]    0    1    9    5
[3,]    0    1    9   11
[4,]    2    5    5   11
```

Example 4: WAP for Multiplication of Matrices

```r
# create a vector of elements
vector1=c(1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16)

# create a matrix with 4* 4 by passing this vector1
matrix1 <- matrix(vector1, nrow = 4, ncol = 4)

# display matrix
print(matrix1)

# create a vector of elements
vector2=c(1,2,3,2,4,5,6,3,4,1,2,7,8,9,4,5)

# create a matrix with 4* 4 by passing  vector2
matrix2 <- matrix(vector2, nrow = 4, ncol = 4)

# display matrix
print(matrix2)
print(" multiplication result")

# multiply matrices
print(matrix1*matrix2)
```

Output:

```
     [,1] [,2] [,3] [,4]
[1,]    1    5    9   13
[2,]    2    6   10   14
[3,]    3    7   11   15
[4,]    4    8   12   16
     [,1] [,2] [,3] [,4]
[1,]    1    4    4    8
[2,]    2    5    1    9
[3,]    3    6    2    4
[4,]    2    3    7    5
[1] " multiplication result"
     [,1] [,2] [,3] [,4]
[1,]    1   20   36  104
[2,]    4   30   10  126
[3,]    9   42   22   60
[4,]    8   24   84   80
```

Example 4: WAP for Division of Matrices.

```r
# create a vector of elements
vector1=c(1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16)

# create a matrix with 4* 4 by passing this vector1
matrix1 <- matrix(vector1, nrow = 4, ncol = 4)

# display matrix
print(matrix1)

# create a vector of elements
vector2=c(1,2,3,2,4,5,6,3,4,1,2,7,8,9,4,5)

# create a matrix with 4* 4 by passing  vector2
matrix2 <- matrix(vector2, nrow = 4, ncol = 4)

# display matrix
print(matrix2)
print(" Division result")

# divide the matrices
print(matrix1/matrix2)
```

Output:

```
     [,1] [,2] [,3] [,4]
[1,]    1    5    9   13
[2,]    2    6   10   14
[3,]    3    7   11   15
[4,]    4    8   12   16
     [,1] [,2] [,3] [,4]
[1,]    1    4    4    8
[2,]    2    5    1    9
[3,]    3    6    2    4
[4,]    2    3    7    5
[1] " Division result"
     [,1]     [,2]      [,3]     [,4]
[1,]    1 1.250000  2.250000 1.625000
[2,]    1 1.200000 10.000000 1.555556
[3,]    1 1.166667  5.500000 3.750000
[4,]    2 2.666667  1.714286 3.200000
```

R – Array: NamingColumns and Rows, AccessingArray Elements, Manipulating Array Elements, CalculationAcross ArrayElements –

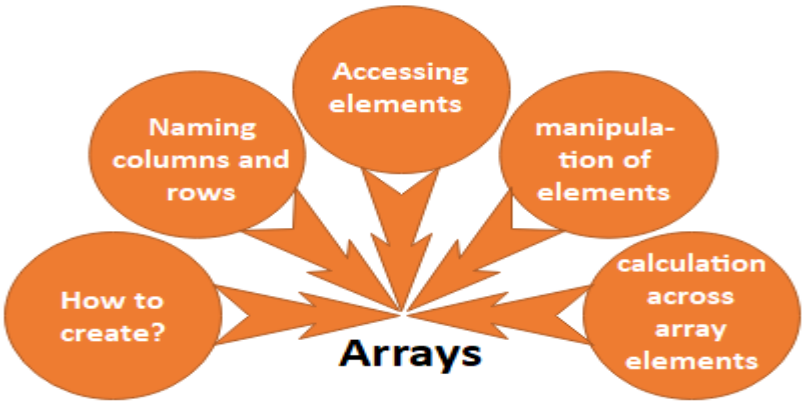
Arrays are essential data storage structures defined by a fixed number of dimensions.

Arrays are used for the allocation of space at contiguous memory locations.

In R Programming Language **Uni-dimensional arrays are called vectors with the length being their only dimension**.

Two-dimensional arrays are called matrices, consisting of fixed numbers of rows and columns.

R Arrays consist of all elements of the same data type.Vectors are supplied as input to the function and then create an array based on the number of dimensions.



Naming rows and columns

In R, we can give the names to the rows, columns, and matrices of the array. This is done with the help of the dim name parameter of the array() function.

It is not necessary to give the name to the rows and columns. It is only used to differentiate the row and column for better understanding.

Below is an example, in which we create two arrays and giving names to the rows, columns, and matrices.

```
#Creating two vectors of different lengths
vec1 <-c(1,3,5)
vec2 <-c(10,11,12,13,14,15)

#Initializing names for rows, columns and matrices
col_names <- c("Col1","Col2","Col3")
row_names <- c("Row1","Row2","Row3")
matrix_names <- c("Matrix1","Matrix2")

#Taking the vectors as input to the array
res <- array(c(vec1,vec2),dim=c(3,3,2),dimnames=list(row_names,col_names,matrix_names))
print(res)
```

OutPut:

```
, , Matrix1

     Col1 Col2 Col3
Row1    1   10   13
Row2    3   11   14
Row3    5   12   15

, , Matrix2

     Col1 Col2 Col3
Row1    1   10   13
Row2    3   11   14
Row3    5   12   15
```

Accessing array elements

The elements are accessed with the help of the index. Simply, we can access the elements of the array with the help of the indexing method.

R Factors:

Factors in R Programming Language are data structures that are implemented to categorize the data or represent categorical data and store it on multiple levels.

They can be stored as integers with a corresponding label to every unique integer.

The R factors may look similar to character vectors, they are integers and care must be taken while using them as strings.

The R factor accepts only a restricted number of distinct values.

For example, a data field such as gender may contain values only from female, male, or transgender.

In the above example, all the possible cases are known beforehand and are predefined. These distinct values are known as levels. After a factor is created it only consists of levels that are by default sorted alphabetically.

Attributes of Factors in R Language

- **x:** It is the vector that needs to be converted into a factor.
- **Levels:** It is a set of distinct values which are given to the input vector x.
- **Labels:** It is a character vector corresponding to the number of labels.
- **Exclude:** This will mention all the values you want to exclude.
- **Ordered:** This logical attribute decides whether the levels are ordered.
- **nmax:** It will decide the upper limit for the maximum number of levels.

Creating a Factor in R Programming Language

The command used to create or modify a factor in R language is – **factor()** with a vector as input.

The two steps to creating an R factor :

- Creating a vector
- Converting the vector created into a factor using function factor()

Examples:1 Let us create a factor gender with levels female, male

```r
# Creating a vector
x <-c("female", "male", "male", "female")
print(x)

# Converting the vector x into a factor
# named gender
gender <-factor(x)
print(gender)
```

Output

```
[1] "female" "male"   "male"   "female"
[1] female male   male   female
Levels: female male
```

Levels can also be predefined by the programmer.

Example 2: Let us create a factor gender with levels female, male and transgender.

```r
# Creating a factor with levels defined by programmer
gender <- factor(c("female", "male", "male", "female"),
          levels = c("female", "transgender", "male"));
gender
```

Output

```
[1] female male   male   female
Levels: female transgender male
```

Further one can check the levels of a factor by using function **levels()**.